CSE3015 – AWS Cloud Practitioner

MODULE 3: AWS Networking, Content Delivery & Security

Comprehensive Exam-Oriented Study Guide

Prepared for Computer Science Engineering Theory & AWS Certification

Table of Contents

1. **PART A — NETWORKING & CONTENT DELIVERY**
    1. Amazon Route 53
    2. Amazon API Gateway
    3. Amazon VPC
    4. VPC Subnets
    5. Route Tables
    6. Security Groups
    7. NAT Gateway
2. **PART B — SECURITY, IDENTIFICATION AND COMPLIANCE**
    1. AWS Identity and Access Management (IAM)
    2. AWS Certificate Manager (ACM)
    3. AWS Key Management Service (KMS)
3. **PART C — COMPARISON TABLES**

PART A — NETWORKING & CONTENT DELIVERY

1. Amazon Route 53

1. Definition

Textbook: Amazon Route 53 is a highly available and scalable cloud Domain Name System (DNS) web service.

Simple Explanation: Think of it as the "phonebook of the internet." Humans read words (like www.google.com), but computers communicate using numbers (IP addresses like 192.0.2.1). Route 53 translates the human-friendly names into computer-friendly IP addresses so traffic can be routed correctly.

2. Why is it needed?

- **Problem:** Memorizing numeric IP addresses for every website is impossible for humans. Moreover, server IPs change frequently in the cloud.
- **Without it:** Users would have to type exact IP addresses into their browsers, and automatic scaling/failover architectures would break when backend IPs change.

3. Core Concept

DNS Concept & Resolution: DNS converts hostnames to IP addresses. When a user types a URL, their machine queries a DNS resolver, which traverses the DNS hierarchy (Root -> TLD -> Authoritative Nameserver) to find the IP.

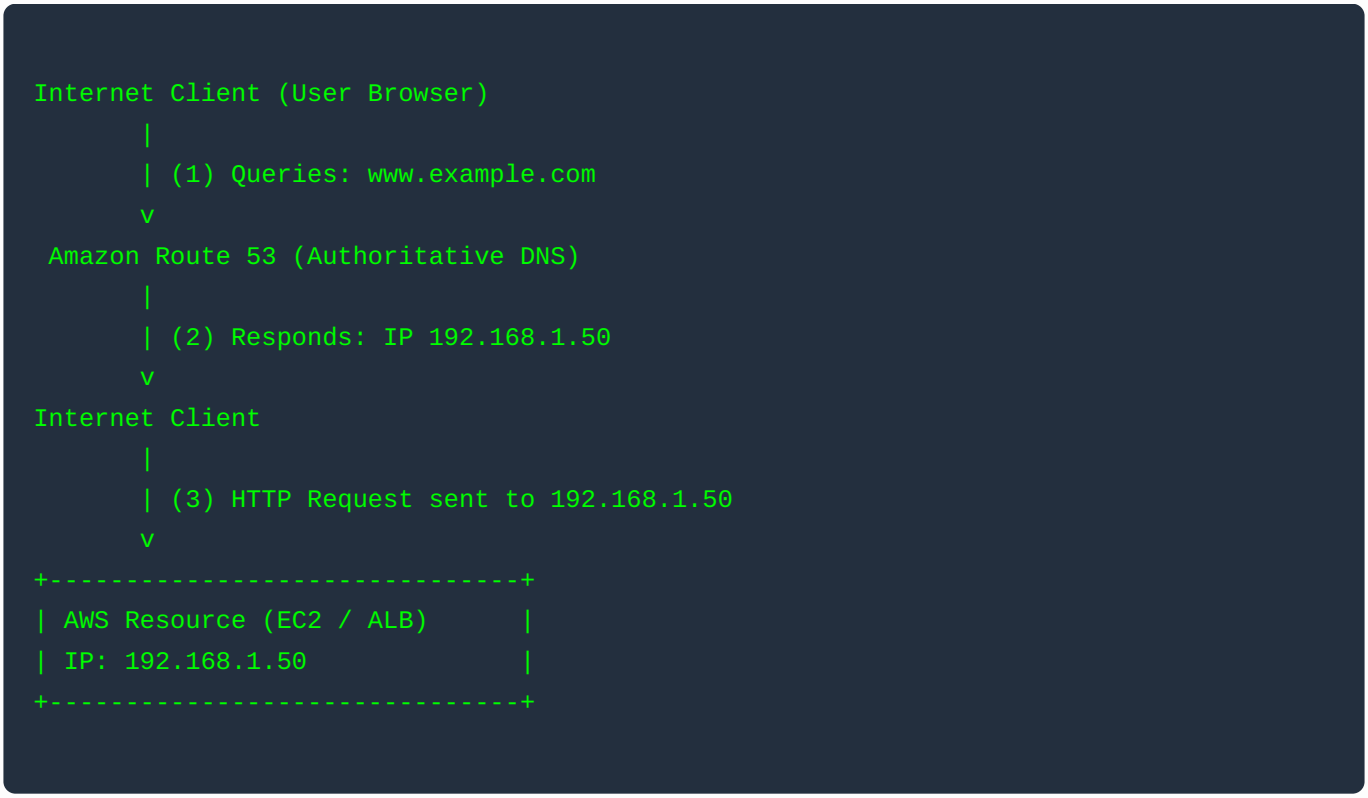
Domain Registration: You can buy and manage domain names directly through Route 53.

4. Architecture / Components

- **Hosted Zones:** A container for records that define how you want to route traffic for a domain.
    - *Public Hosted Zone:* Controls routing on the internet (e.g., example.com).
    - *Private Hosted Zone:* Controls routing within an Amazon VPC (e.g., internal-db.local).

- **DNS Records:** Instructions on how to route traffic.
    - **A Record:** Maps hostname to IPv4 address.
    - **AAAA Record:** Maps hostname to IPv6 address.
    - **CNAME:** Maps a hostname to another hostname (cannot be used for the root domain).
    - **Alias Record:** AWS-specific extension. Maps a hostname to an AWS resource (like an ALB or CloudFront distribution). Unlike CNAME, it works on the root domain and is free.
    - **NS Record:** Name Server record; delegates a DNS zone to use given authoritative name servers.
    - **SOA Record:** Start of Authority; contains administrative info about the zone.
- **TTL (Time To Live):** How long DNS resolvers should cache the record.
- **Health Checks:** Route 53 monitors your applications and can divert traffic if an endpoint is unhealthy.

5. Architecture Diagram: DNS Resolution



6. Routing Policies (Crucial for Exam)

- **Simple Routing:** Standard DNS record with no special logic. Returns a single IP (or multiple IPs randomly).
- **Weighted Routing:** Routes traffic in proportions (e.g., 80% to Server A, 20% to Server B). Great for A/B testing.

- **Latency-based Routing:** Routes traffic to the region with the lowest network latency for the user.
- **Failover Routing:** Used for Active-Passive disaster recovery. If Primary fails health check, traffic routes to Secondary.
- **Geolocation Routing:** Routes traffic based on the geographic location of the user (e.g., EU users go to EU servers).
- **Geoproximity Routing:** Routes traffic based on geographic location of resources, allowing shift in traffic sizes via "bias".
- **Multivalue Answer Routing:** Returns multiple IP addresses but only for *healthy* endpoints. It acts as a simple client-side load balancer.

7. AWS Console / Practical Usage

1. Open AWS Console and search for **Route 53**.
2. Click **Hosted zones** -> **Create hosted zone**.
3. Enter Domain Name (e.g., myexamapp.com) and select Public Hosted Zone.
4. Click **Create record**.
5. Choose Routing Policy (e.g., Simple routing).
6. Configure the record: Type = A, Value = [EC2 IP address], TTL = 300 seconds.
7. Save. Test by pinging the domain name in the terminal.

> **IMPORTANT FOR EXAM:** Route 53 ALIAS records are better than CNAMEs for AWS resources. ALIAS can map the zone apex (naked domain, e.g., domain.com) to an ALB, while CNAME cannot. ALIAS queries to AWS resources are also free.

8. Advantages & Limitations

Advantages: Global service (not tied to a region), highly reliable (100% SLA), integrates seamlessly with AWS resources, provides health checks.

Limitations: DNS propagation can take time (depending on TTL), advanced routing requires careful configuration to avoid loops.

9. Scenario-Based Example

Scenario: You are launching a new version of your web app but want to test it with only 10% of your user base before a full rollout. Which AWS service and feature should you use?

Answer: Use **Amazon Route 53** with a **Weighted Routing Policy**. Assign a weight of 90 to the old application endpoint and a weight of 10 to the new application endpoint.

2. Amazon API Gateway

1. Definition

Textbook: Amazon API Gateway is a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale.

Simple Explanation: It acts as a "front door" for applications to access data, business logic, or functionality from your backend services. Instead of exposing your servers directly to the internet, clients talk to the API Gateway, which forwards the request safely.

2. Why is it needed?

Without API Gateway, developers would have to manually build infrastructure to handle authentication, rate limiting (throttling), CORS, and scaling for their backend endpoints. API Gateway handles all this heavy lifting.

3. Core Concept

- **API (Application Programming Interface):** A set of rules allowing different software entities to talk to each other.
- **REST API:** Feature-rich, highly customizable API style over HTTP.
- **HTTP API:** A newer, faster, cheaper, and simpler version of REST APIs, designed mainly for serverless backends.
- **WebSocket API:** Maintains a persistent, two-way connection (ideal for real-time chat apps).

4. Architecture / Components

- **Resources:** The path of the URL (e.g., `/users` or `/orders`).
- **Methods:** HTTP actions applied to resources (e.g., GET, POST, PUT, DELETE).
- **Integration:** What happens when the API is called (e.g., trigger a Lambda function, call an EC2 instance).
- **Stages:** Environments like `dev`, `test`, or `prod`. A deployment must be associated with a stage to be accessible.
- **Throttling:** Limiting requests (e.g., 1000 requests per second) to prevent backend overload.
- **Caching:** Storing API responses to reduce backend load and improve latency.

5. Architecture Diagram



6. Working / Flow

1. Client sends a REST request (e.g., GET /products) to the API Gateway endpoint.
2. API Gateway authenticates the request (via IAM, Cognito, or custom authorizer).
3. It checks throttling limits. If limits are exceeded, returns HTTP 429 (Too Many Requests).
4. It routes the request to the backend integration (e.g., Lambda).
5. Backend processes data and returns it to API Gateway.
6. API Gateway transforms the response (if configured) and sends it to the client.

7. AWS Console / Practical Usage: API + Lambda

1. Create an AWS Lambda function first (e.g., "HelloWorldLambda").
2. Open API Gateway in the AWS Console.
3. Click **Create API** -> Choose **REST API**.
4. Create a Resource (e.g., `/hello`).
5. Create a Method (e.g., `GET`) under that resource.
6. Set the Integration Type to **Lambda Function** and select "HelloWorldLambda".
7. Click **Deploy API**, create a new Stage (e.g., `prod`).
8. Copy the Invoke URL and test in a browser.

> **PRACTICAL TIP:** API Gateway + AWS Lambda + DynamoDB is the standard "Serverless 3-Tier Architecture."

8. Advantages & Limitations

Advantages: Serverless (no infrastructure to manage), built-in DDoS protection, handles API versioning (stages), caching improves speed.

Limitations: Max timeout for integrations is 29 seconds (Lambda hard limit). If your task takes 30 seconds, API Gateway will drop the connection.

3. Amazon VPC (Virtual Private Cloud)

> **CRITICAL TOPIC:** VPC is the foundation of AWS networking. You must understand every component here for the exam.

1. Definition

Textbook: Amazon VPC lets you provision a logically isolated section of the AWS Cloud where you can launch AWS resources in a virtual network that you define.

Simple Explanation: It is your own private, secure datacenter inside the AWS public cloud. You control the IP addresses, subnets, and routing.

2. Why is it needed?

Without a VPC, every EC2 instance would sit on the public internet, exposed to hackers. A VPC provides isolation, allowing you to hide databases and backend servers while only exposing web servers.

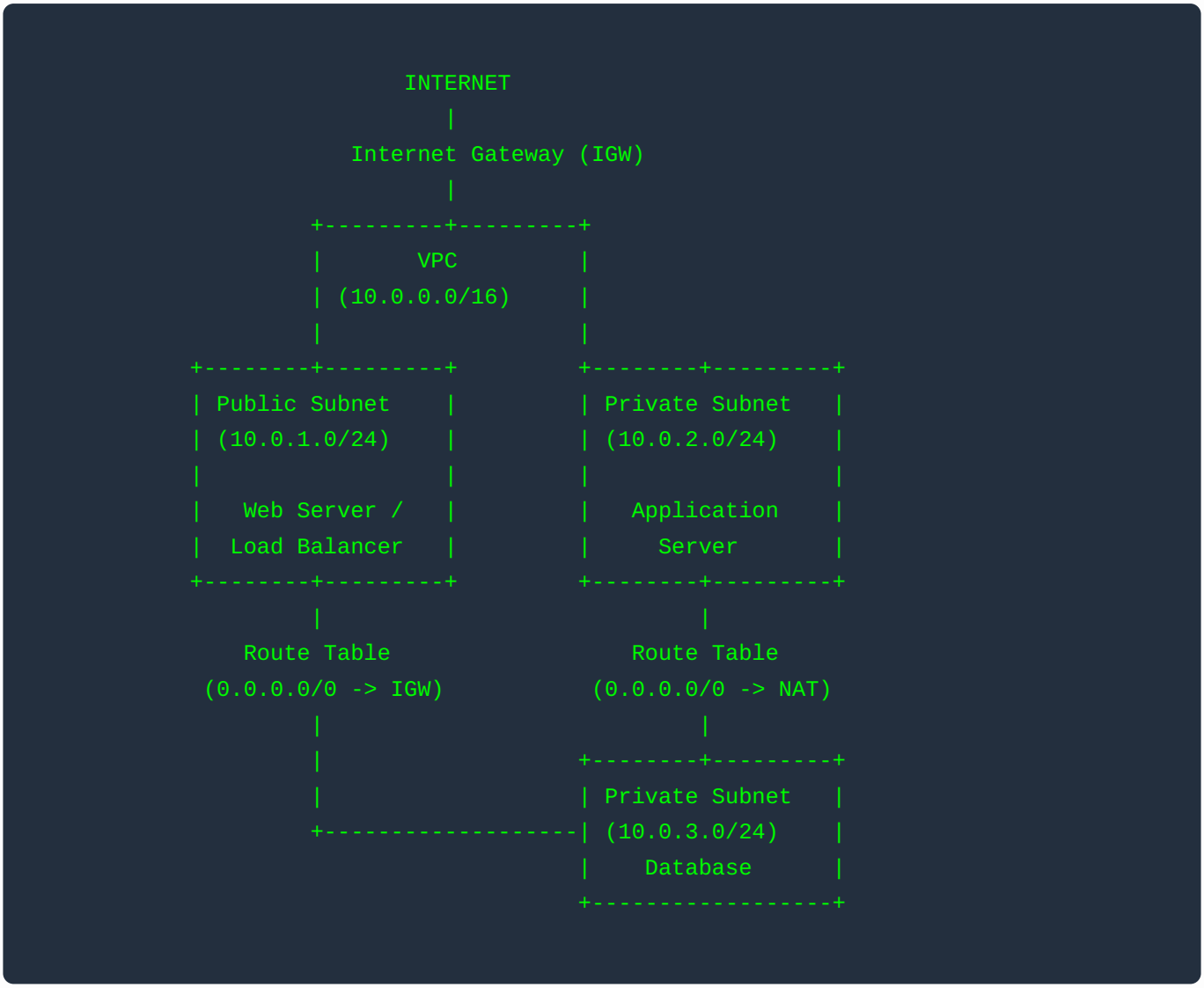
3. Core Concept

- **Default VPC:** Created automatically in every AWS account per region. Pre-configured to allow immediate resource deployment. All subnets are public by default.
- **Custom VPC:** Created manually by you. Nothing is allowed by default; it is highly secure but requires manual setup of components.
- **CIDR (Classless Inter-Domain Routing):** A method of assigning IP addresses. E.g., `10.0.0.0/16` means the network has 65,536 IP addresses.

4. Architecture / Components

- **VPC:** The outer boundary covering an entire AWS Region.
- **Internet Gateway (IGW):** A scalable, redundant component attached to the VPC that allows communication between instances in your VPC and the internet.
- **Subnets:** Chunks of the VPC IP range isolated to specific Availability Zones.
- **Route Tables:** A set of rules (routes) used to determine where network traffic is directed.
- **Security Groups & NACLs:** Virtual firewalls.

5. Complete Architecture Diagram: Web Application VPC



6. Scenario-Based Example

Scenario: You are designing a 3-tier web application. Where do you place the components?

Answer: 1. **Presentation Tier (ALB/Web Servers):** Place in a **Public Subnet** because they need direct internet access to serve users. 2. **Logic Tier (App Servers):** Place in a **Private Subnet**. They only receive traffic from the Web Servers, not the internet. 3. **Data Tier (RDS DB):** Place in a **Deep Private Subnet**. It only receives traffic from the App Servers. This minimizes the attack surface.

4. VPC Subnets

1. Definition

Textbook: A subnet is a range of IP addresses in your VPC. You launch AWS resources into a specified subnet.

Simple Explanation: If a VPC is a large building, a subnet is a specific room inside that building. Subnets divide the large VPC network into smaller, manageable chunks.

2. Core Concept

- **CIDR Blocks:** Subnet CIDRs must be a subset of the VPC CIDR (e.g., VPC is /16, subnets are /24).
- **Availability Zone (AZ) Relationship:** A subnet must reside entirely within **ONE** Availability Zone. It cannot span multiple AZs. (Crucial for high availability design).

3. Public vs Private Subnets

What makes a subnet public?

> **COMMON MISTAKE:** Just naming a subnet "Public-Subnet-1" does NOT make it public. A subnet is public **ONLY IF** its associated Route Table has a route directing internet-bound traffic (0.0.0.0/0) to an Internet Gateway (IGW).

- **Public Subnet:** Has a route to the Internet Gateway. Resources inside can get public IP addresses and talk to the internet.
- **Private Subnet:** Does NOT have a route to the IGW. Resources inside cannot be reached directly from the internet.

4. Reserved IPs

AWS reserves **5 IP addresses** in every subnet for internal networking purposes (Network address, VPC router, DNS, Future use, Broadcast). You cannot use them for EC2 instances.

5. Route Tables

1. Definition

Textbook: A route table contains a set of rules, called routes, that determine where network traffic from your subnet or gateway is directed.

Simple Explanation: It acts as the GPS or traffic cop for a subnet, telling packets exactly which door to exit through to reach their destination.

2. Components

- **Destination:** The target IP range the traffic wants to go to.
- **Target:** The gateway or connection through which to send the traffic (e.g., Local, IGW, NAT).

Main Route Table: Automatically created with the VPC. Controls routing for all subnets that are not explicitly associated with another route table.
Custom Route Table: Created manually to isolate routing logic (e.g., keeping private subnets private).

3. Routing Examples

Subnet Type	Destination	Target	Meaning
Public Subnet	10.0.0.0/16	local	Traffic to other resources in the VPC stays inside the VPC.
Public Subnet	0.0.0.0/0	igw-1a2b3c	Any traffic bound for the internet goes to the Internet Gateway.
Private Subnet	0.0.0.0/0	nat-9f8e7d	Any internet-bound traffic goes to the NAT Gateway (to hide private IP).

6. Security Groups

1. Definition

Textbook: A security group acts as a virtual firewall for your EC2 instances to control incoming and outgoing traffic.

Simple Explanation: It is a bouncer at the door of your specific EC2 server, checking if traffic is allowed in or out based on rules you define.

2. Core Concept: Stateful Firewall

> **IMPORTANT FOR EXAM:** Security Groups are **STATEFUL**. This means if you allow an incoming request (e.g., HTTP on port 80), the return response is automatically allowed to flow out, regardless of outbound rules. Network ACLs are **STATELESS** (you must explicitly allow return traffic).

3. Rules configuration

- **Default Inbound Rule:** DENY ALL. You must explicitly add rules to let traffic in.
- **Default Outbound Rule:** ALLOW ALL. By default, instances can send traffic anywhere.

4. Practical Examples

```
Rule Name | Protocol | Port Range | Source
------------------------------------------------
HTTP      | TCP      | 80         | 0.0.0.0/0 (Internet)
HTTPS     | TCP      | 443        | 0.0.0.0/0 (Internet)
SSH       | TCP      | 22         | 203.0.113.1/32 (My Office IP only)
MySQL     | TCP      | 3306       | sg-0abcd123 (Web Server SG only)
```

Why shouldn't SSH be 0.0.0.0/0? Because that opens administrative access to the entire globe, leading to immediate brute-force attacks. Always restrict SSH (Port 22) to your specific IP address.

Security-group-based rules: You can allow traffic from *another security group* instead of an IP address. E.g., The Database SG allows port 3306 ONLY from the Web Server SG. This is highly secure and dynamic.

7. NAT Gateway

1. Definition

Textbook: Network Address Translation (NAT) Gateway is a managed service that allows resources in a private subnet to connect to the internet while preventing the internet from initiating a connection to those instances.

2. Why is it needed?

Your database in a private subnet needs to download security patches from the internet. However, because it is in a private subnet, it has no public IP and no direct route to the Internet Gateway. A NAT Gateway solves this.

3. Architecture & Flow

1. NAT Gateway is placed in the **Public Subnet** (must have an Elastic IP).
2. Private instance sends internet-bound traffic to the NAT Gateway (configured via private route table).
3. NAT Gateway replaces the instance's private IP with its own public Elastic IP.
4. Traffic goes to IGW -> Internet.
5. Response returns to NAT Gateway, which translates the IP back and forwards it to the private instance.

```
Private EC2 (10.0.2.50)
       |
Private Route Table (0.0.0.0/0 -> NAT-GW)
       |
       v
NAT Gateway (Public Subnet, EIP: 198.51.100.2)
       |
       v
Internet Gateway (IGW)
       |
       v
Internet (Patch Server)
```

4. Advantages & Limitations

Advantages: Fully managed by AWS, highly available, scales bandwidth automatically (up to 45 Gbps), zero administration.

Limitations: Charges apply per hour and per GB of data processed (can get expensive). Must be created in a specific AZ (for HA across AZs, you need multiple NAT Gateways).

PART B — SECURITY, IDENTIFICATION AND COMPLIANCE

8. AWS Identity and Access Management (IAM)

1. Definition

Textbook: AWS IAM allows you to manage access to AWS services and resources securely. It defines *who* is authenticated and *what* they are authorized to do.

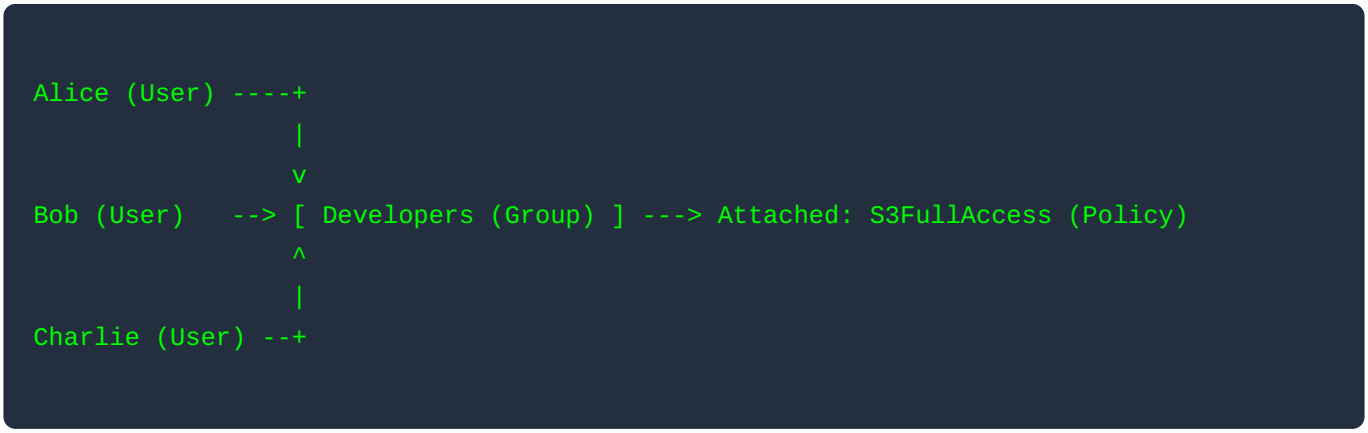
Simple Explanation: IAM is the bouncer for your entire AWS account. It determines who gets an ID card (User), who belongs to which department (Group), what keys they hold (Policies), and temporary visitor passes (Roles).

2. Core Components

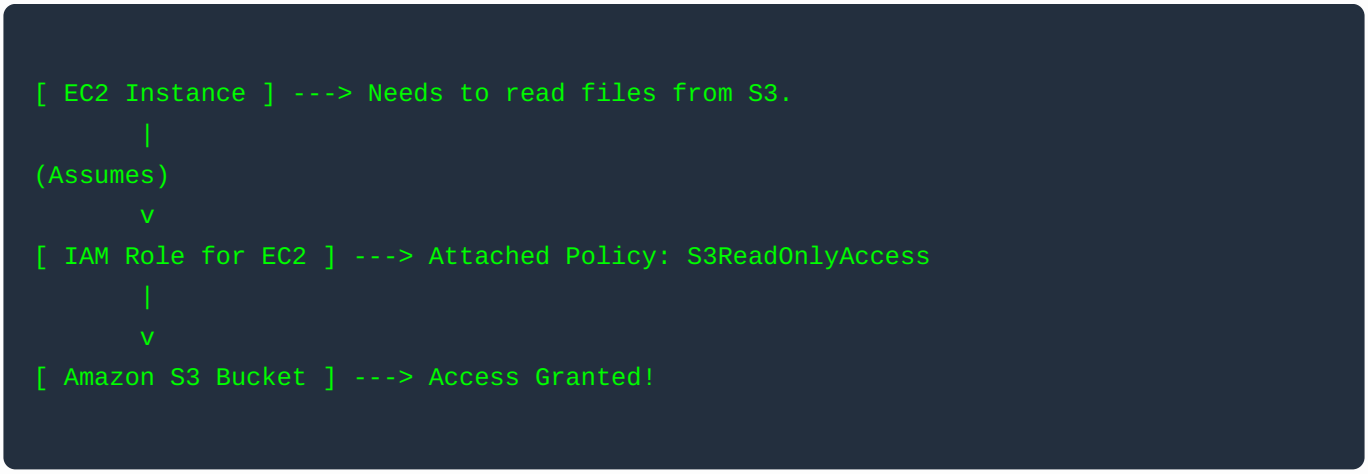
- **AWS Account Root User:** The email used to create the account. Has absolute, unrestricted power. *Exam tip: Never use it for daily tasks. Lock it away with MFA.*
- **Users:** Physical people or specific applications. They have permanent credentials (password or Access Keys).
- **Groups:** A collection of users. E.g., 'Developers' group. You attach a policy to the group, and all users inside inherit it.
- **Roles:** Temporary credentials assumed by trusted entities (like EC2 instances, Lambda functions, or federated users). They do not have passwords.
- **Policies:** JSON documents that define permissions (Allow or Deny).

3. IAM Architecture Diagrams

Group inheritance flow:



EC2 assuming a Role:



Why use a role for EC2? If you put a User's Access Keys inside EC2 code, hackers can steal them. Roles provide rotating, temporary credentials safely injected by AWS.

4. IAM Policy JSON Breakdown

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ec2:StartInstances",
      "Resource": "arn:aws:ec2:us-east-1:123456789012:instance/i-0abcd1234"
    }
  ]
}
```

- **Version:** The language version of the policy. (Always `2012-10-17` for current syntax).
- **Statement:** Container for the rules.
- **Effect:** Can only be `Allow` or `Deny`. (Explicit Deny always overrides an Allow).
- **Action:** The specific API call allowed. Format is `service:operation`. Here, we allow starting an EC2 instance.
- **Resource:** The exact AWS resource the action applies to, specified by ARN (Amazon Resource Name).

5. Authentication vs Authorization

Authentication (AuthN): Proving *who* you are (e.g., logging in with a Username + Password + MFA).

Authorization (AuthZ): Determining *what* you are allowed to do once logged in (evaluated via IAM Policies).

> **PRINCIPLE OF LEAST PRIVILEGE:** Always grant only the minimum permissions required to perform a task. Don't give `AdministratorAccess` if someone only needs to read an S3 bucket.

9. AWS Certificate Manager (ACM)

1. Definition

Textbook: ACM is a service that lets you easily provision, manage, and deploy public and private Secure Sockets Layer/Transport Layer Security (SSL/TLS) certificates.

Simple Explanation: ACM provides the "padlock" icon in your web browser (HTTPS). It encrypts data traveling between your user's browser and your AWS servers (Encryption in Transit).

2. Why is it needed?

Without SSL/TLS, data sent over HTTP is in plain text. Hackers on a public Wi-Fi network could read passwords or credit card numbers. Certificates prove the identity of your website and encrypt the connection.

3. Architecture / Flow

```
User Browser
       |
       |  (HTTPS: Encrypted tunnel)
       v
Application Load Balancer (ALB)  <--- [ ACM Certificate Attached Here ]
       |
       |  (HTTP: Decrypted, internal VPC routing)
       v
EC2 Web Servers
```

4. Core Concepts

- **Public Certificates:** Free in AWS, auto-renewed, used for public websites facing the internet.
- **Validation:** To get a certificate, you must prove you own the domain via *DNS Validation* (adding a CNAME record to Route 53) or *Email Validation*.
- **Integration:** ACM certificates cannot be installed directly on EC2 instances. They are attached to ALBs, API Gateway, or CloudFront distributions.

10. AWS Key Management Service (KMS)

1. Definition

Textbook: AWS KMS makes it easy for you to create and manage cryptographic keys and control their use across a wide range of AWS services.

Simple Explanation: While ACM encrypts data *while it moves* (in transit), KMS encrypts data *where it is stored* (at rest, like on a hard drive or inside a database).

2. Envelope Encryption (Crucial Concept)

KMS does not encrypt massive gigabytes of data directly. It uses **Envelope Encryption**. It generates a Data Encryption Key (DEK) to encrypt your large file. Then, KMS uses a Master Key to encrypt the DEK itself.



How Decryption works: AWS sends the Encrypted DEK to KMS. KMS checks permissions, decrypts it using the Master Key, and returns the plaintext DEK. The application uses the plaintext DEK to unlock the 10 GB file.

3. Symmetric vs Asymmetric

- **Symmetric:** Same key used to encrypt and decrypt. (Default and most common in KMS).
- **Asymmetric:** Public key to encrypt, Private key to decrypt. (Used when you need to send keys outside AWS).

4. Practical Example in AWS

When creating an EBS Volume or an RDS Database, you simply check a box: `[x] Enable Encryption` and select a KMS key (default `aws/ebs` or `aws/rds`). AWS handles the envelope encryption automatically in the background. It is seamless.

PART C — COMPARISON TABLES

1. Public vs Private Subnet

Feature	Public Subnet	Private Subnet
Internet Access	Direct (via IGW)	Indirect (via NAT Gateway)
Public IPs	Instances get Public IPs	Instances only have Private IPs
Route Table	0.0.0.0/0 -> igw-xxx	0.0.0.0/0 -> nat-xxx
Use Case	Web Servers, Load Balancers	Databases, App Servers

2. Internet Gateway (IGW) vs NAT Gateway

Feature	Internet Gateway (IGW)	NAT Gateway
Purpose	Allows public subnets to reach internet	Allows private subnets to reach internet
Inbound Traffic	Allows internet to initiate connections	Blocks inbound internet connections
Cost	Free	Hourly charge + data processing fee
Scope	One per VPC	Resides in a specific AZ

3. Security Group vs Network ACL

Feature	Security Group (SG)	Network ACL (NACL)
Level	Instance level (EC2)	Subnet level
Statefulness	Stateful (return traffic auto-allowed)	Stateless (must allow both inbound/outbound)
Rules allowed	ALLOW rules only	ALLOW and DENY rules
Evaluation	All rules evaluated before decision	Rules evaluated in number order (lowest first)

4. Route 53 vs Traditional DNS

Feature	Amazon Route 53	Traditional DNS
Integration	Deeply integrated with AWS (ALIAS records)	Requires standard CNAME/A mapping
Traffic Flow	Advanced routing (Latency, Geo, Weighted)	Usually just simple routing (Round Robin)
Health Checks	Native health checks and failover	Requires external monitoring tools

5. API Gateway vs Load Balancer (ALB)

Feature	API Gateway	Application Load Balancer
Primary Use	REST/Serverless APIs (Lambda integrations)	Routing traffic to EC2/Containers
Features	Throttling, Caching, API Keys, Stages	Path-based routing, target groups
Pricing	Per million API calls	Per hour + LCU (data processed)

6. IAM User vs IAM Role

Feature	IAM User	IAM Role
Credentials	Permanent (Password, Access Keys)	Temporary (STS generated)
Who uses it?	Physical humans, external apps	AWS Services (EC2, Lambda), Federated users
Security Risk	High (keys can be leaked/stolen)	Low (keys auto-rotate and expire)

7. ACM vs KMS

Feature	AWS Certificate Manager (ACM)	AWS Key Management Service (KMS)
Protects	Data in Transit (moving over network)	Data at Rest (stored on disk/DB)
Technology	SSL/TLS Certificates (Asymmetric)	Encryption Keys (Symmetric DEK/ CMK)
Integration point	Load Balancers, CloudFront, API GW	S3, EBS, RDS, DynamoDB

8. Authentication vs Authorization

Feature	Authentication (AuthN)	Authorization (AuthZ)
Question	Who are you?	What are you allowed to do?
Mechanism	Password, MFA token, Biometrics	IAM Policies (JSON documents)
Failure Result	401 Unauthorized (Login failed)	403 Forbidden (Access denied)

> **END OF MODULE 3 STUDY GUIDE.** Best of luck on your university examinations and AWS Cloud Practitioner certification!